

Software Project Management – IS212

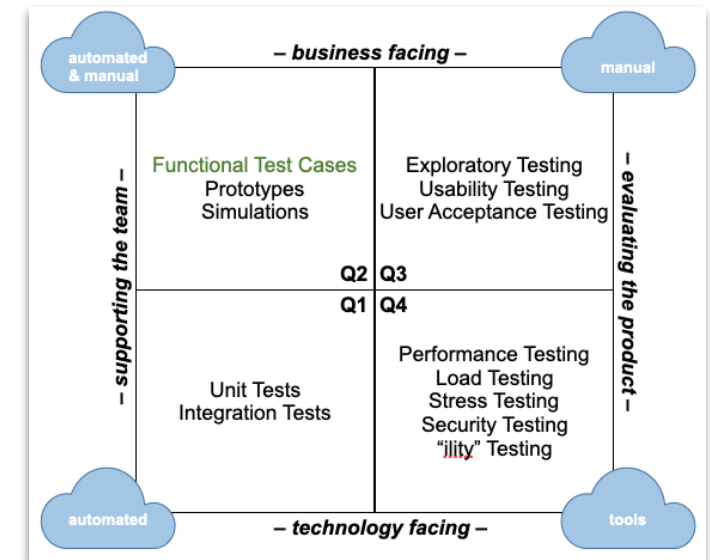
Communicating Design & Collaborating on Code

just-enough UML, prompt scaffolding, & Git workflows

Recap

Week 4 Recap – Agile Testing & Test Cases

- How does testing fit into an agile software development process?
- Which tests to cover in the agile testing quadrant?
- How do we derive test cases from ACs?
- What's a happy, negative, and boundary test?
- When do we write tests in scrum?
- Why are regression testing and test traceability important?



Week 4 Recap –Software Architecture

- Why do we seek “just enough” architecture?
- The C4 model zooms from **Context** → **Containers** → **Components** → **Code**
 - *What does each level focus on?*
- Which C4 levels do most teams draw – and which only when they add value?
- Why do we prefer a text DSL (Structurizr, PlantUML, Mermaid) over a flat exported image?

Today we “**zoom in**” to C4-Code

Today: Communicating Design & Collaborating on Code

Objectives

- Read & draw “just enough” UML class and sequence diagrams (C4 “Code” level models)
- Use UML diagrams as shared design and as context for AI-assisted code
- Collaborate with Git using branches, pull requests, and a clear branching strategy
- Compare feature-based vs trunk-based development and adapt when teammates include AI agents

Outline

- **Part 1 – Just Enough Code Modelling:** class & sequence diagrams, the GP-surgery example, modelling with AI
- **Part 2 – Git as a Collaboration System:** Git & pull requests, branching strategies, collaborating with AI agents

UML Class & Sequence Diagrams

– *visualising **code-level** design* –

From C4 Components to Code-Level Models

- Last week, we modelled software at the Context, Container, and Component levels
- Today, we “zoom in” to the **code inside a component**: its classes, objects, and interactions
- We will use two lightweight **UML** views:
 - **Class diagrams**: *what objects/classes exist and how they relate*
 - **Sequence diagrams**: *how those objects interact over time*
- C4 does not prescribe a Code-level notation; UML is simply **one useful choice**
- In agile development, model **only** when the diagram **communicates something more clearly than the code itself**
- Keep useful diagrams as **diffable text** (e.g. **PlantUML** / **Mermaid**) so they can evolve with the code

Example – A UML Class Diagram

- A **class diagram** is a model of your code – here, two Python classes and an “is-a” relationship

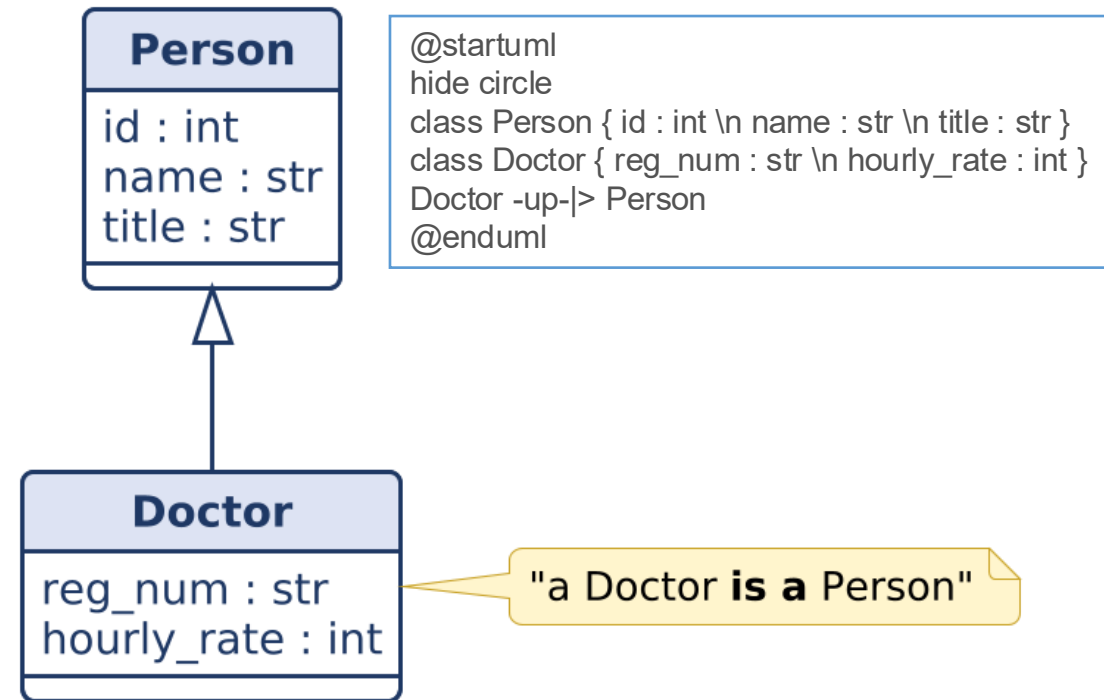
```
class Person(db.Model):
    __tablename__ = 'person'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    title = db.Column(db.String(10))

class Doctor(Person):
    __tablename__ = 'doctor'

    id = db.Column(db.Integer, db.ForeignKey('person.id'),
    reg_num = db.Column(db.String(15))
    hourly_rate = db.Column(db.Integer)
```

Python/Flask code

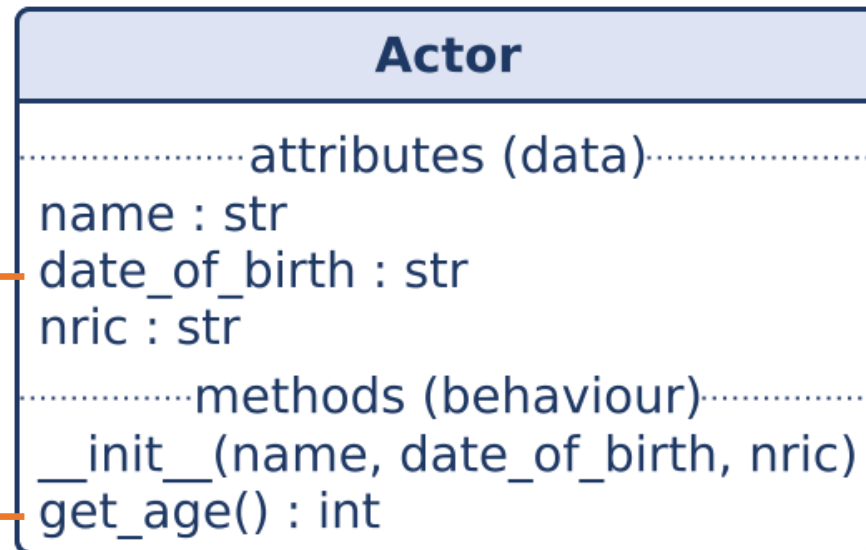


A model of the code

OO Basics – Recap

- **Abstraction** – model only the characteristics that matter; hide the rest
- A **class** is a template; an **object** is an instance created from it
- **Encapsulation** bundles data with the methods that operate on it; `__init__` is (roughly) the constructor
- In a class box: **name** on top, **attributes** (data) in the middle, **methods** (behaviour) below

The method **calculates** the actor's age using the string value of their date of birth



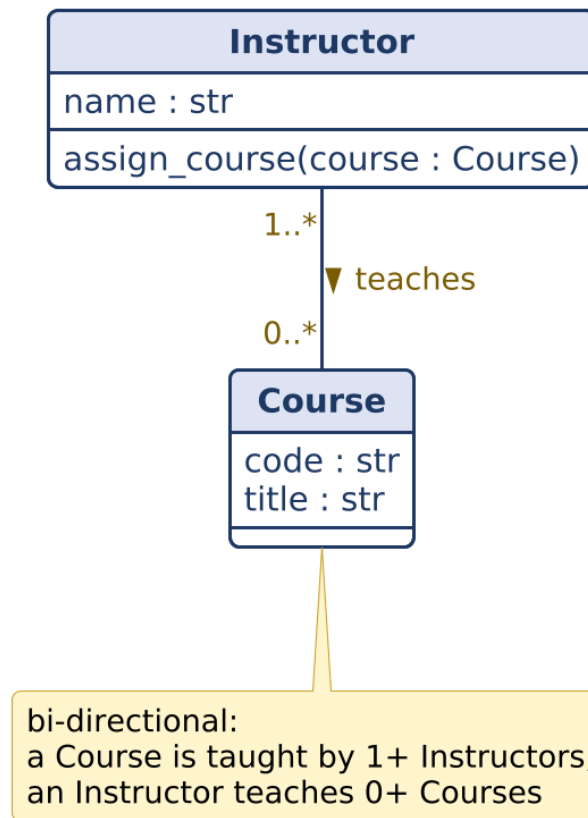
data + the methods that operate on it, bundled together

Association – Multiplicity & Direction

- In class diagrams, **associations** capture how objects of two classes **are connected** (if at all)
- ***Multiplicity*** – one-to-one, one-to-many, many-to-many, ...?
 - E.g. a Doctor has **many** Patients, but a Patient has **one** Doctor
- ***Directionality*** – is the association uni- or bidirectional?
 - E.g. Invoice → Customer (**one-way**), Department ↔ Employee (**mutual awareness**)

Association – Multiplicity & Direction

- The number at each end says how many of **that** class take part: a **Course** is taught by **1..*** **Instructors**; an **Instructor** teaches **0..*** **Courses**

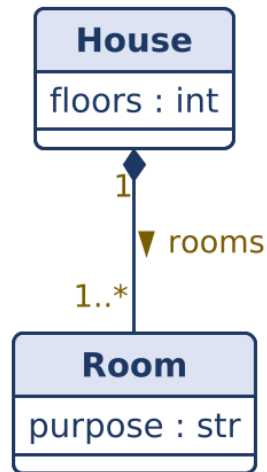


*Why do the **multiplicities** differ
in the two directions?*

Composition (and Aggregation)

Two special “whole–part” associations

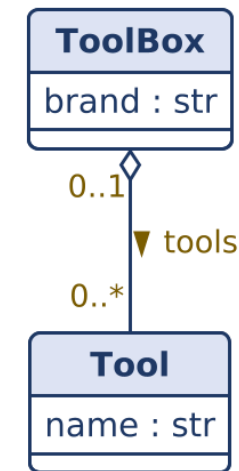
Composition – “x consists of y”; y **cannot** exist without x (a **House** consists of **Rooms**); filled diamond ◆



Composition (consists-of):
a Room cannot exist without its House

Aggregation – “x has a y”, y exists independently; hollow diamond ◇

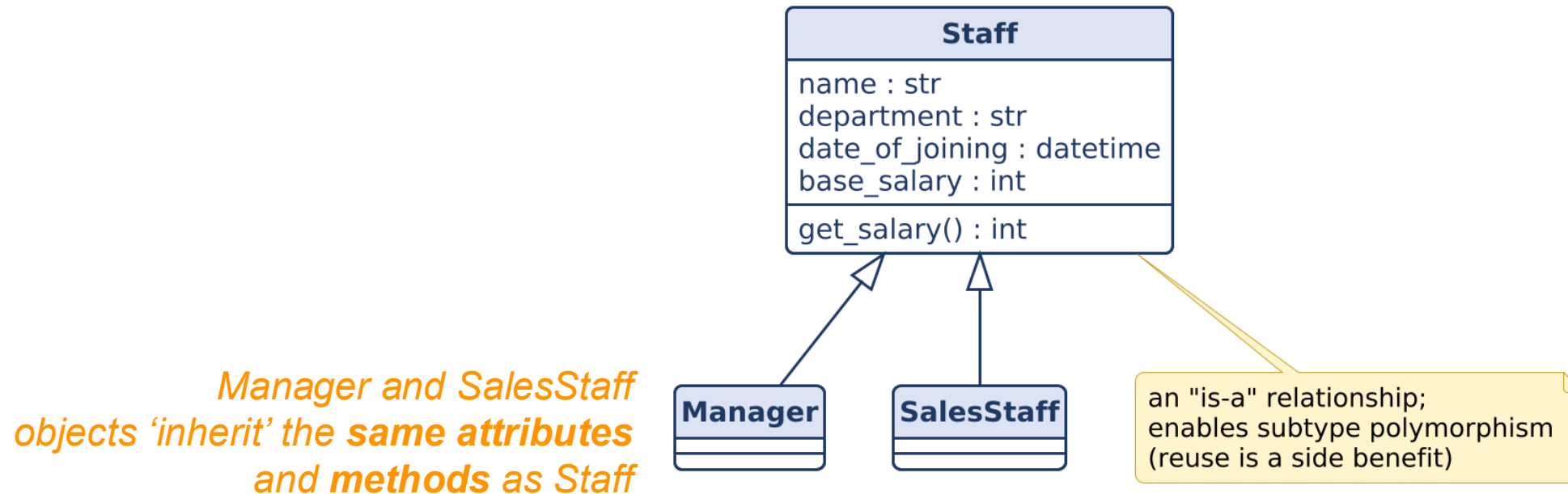
(Aggregation is *rarely necessary* – a plain association usually communicates the same thing as clearly)



Aggregation (has-a): a Tool can exist outside of a ToolBox

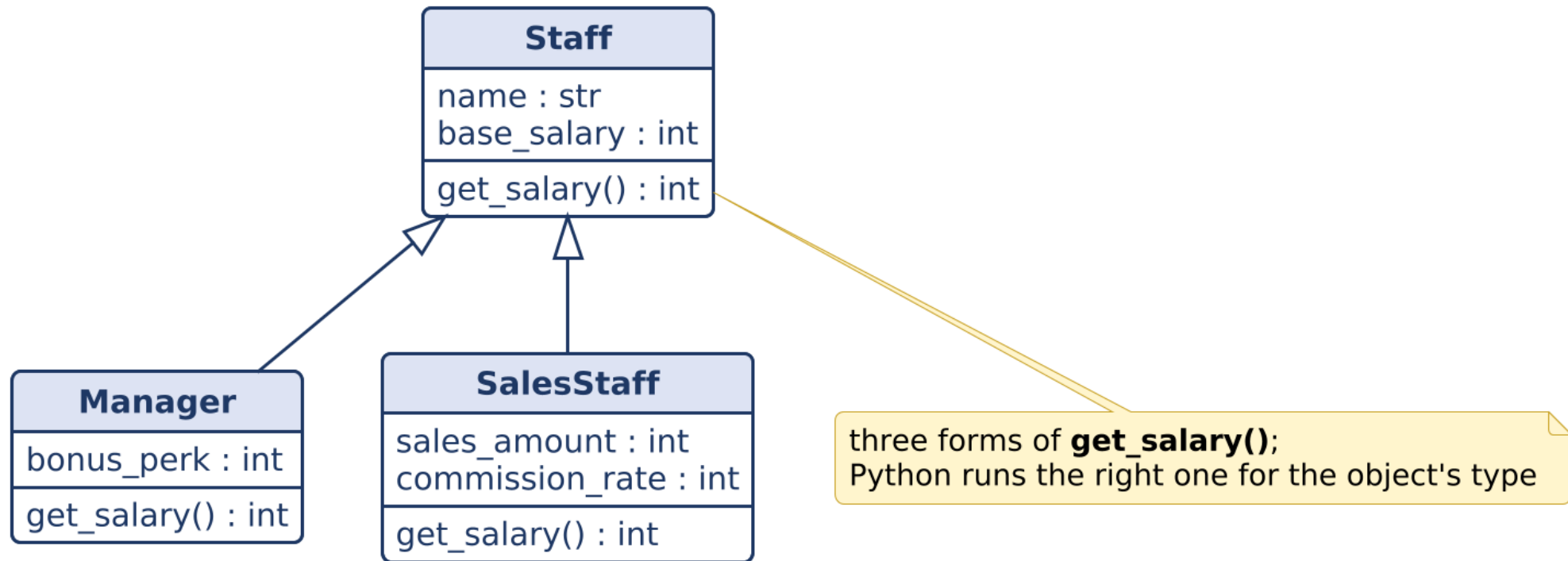
Inheritance

- **Inheritance** models an “is-a” relationship and enables subtype **polymorphism**



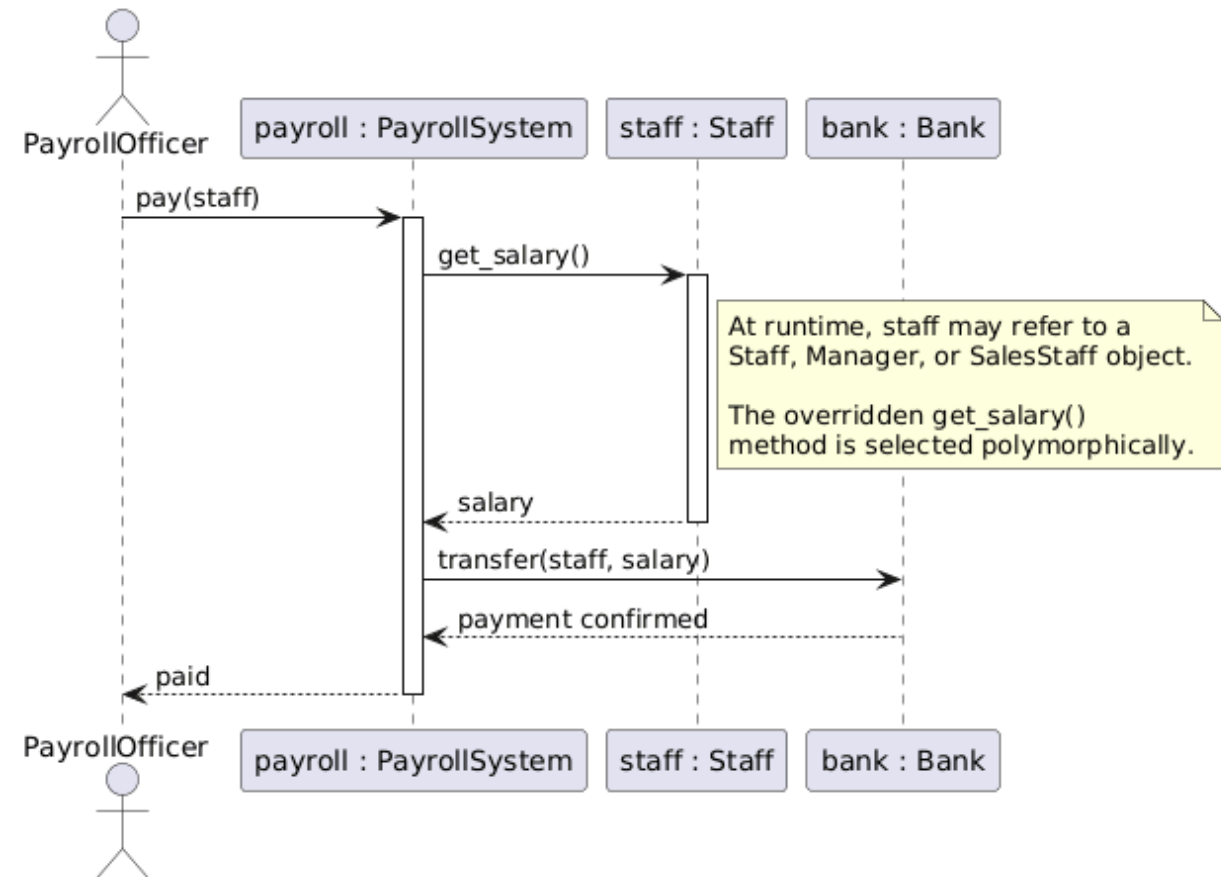
Polymorphism

- **Polymorphism** —a subclass can **override** a method (e.g. `get_salary()`); Python runs the one matching the object's actual type



UML Sequence Diagrams

- A **sequence diagram** shows how participants interact over time to carry out one scenario
- **Participants** appear across the top; each has a vertical lifeline
- **Time flows downward**; arrows show messages or method calls between participants
- The diagram **focuses on order and interaction**, complementing a class diagram's view of structure
- **Example:** a Payroll Officer asks the Payroll System to pay a Staff member; the system obtains the salary polymorphically, transfers it through the Bank, and confirms payment



UML in the AI Era

- **Code → Diagram:** use AI to **reverse-engineer** structure and interactions for **understanding and review**
- **Diagram → Code:** use class and sequence diagrams as **compact specs** to constrain what an agent builds when the design **would otherwise be ambiguous**
- **Plausible ≠ correct:** AI-generated diagrams may still get relationships, multiplicities, composition, or ordering wrong
- **Human judgement remains essential:** you decide whether the model **actually** captures the intended design
- Keep diagrams as text (**PlantUML** / **Mermaid**): versionable, diffable, reviewable, and easy to regenerate as the code evolves

Worked Example

user story → UML diagrams → prompt scaffolding

Example Briefing – GP Surgery System

- A **clinic** runs **tele-consults** and wants a custom consultation & billing app
- Some excerpts from the customer briefing:
 - “Each **patient** tops up a clinic **e-wallet** they spend on **consultations**...”
 - “The **doctor** records the **diagnosis**, **prescription**, and **consultation type** (standard / long / follow-up)...”
 - “each **type** has a set **fee**; on submit the record is saved and the fee is **charged to the e-wallet**...”
 - “if the wallet is short, the visit is **still recorded** – the **charge** is just left outstanding.”



Bold nouns = potential domain classes or attributes

A User Story Card

As a **consulting doctor**, I want to record a patient's diagnosis, prescription, and consultation type so that the consultation is documented in the patient's record.

Acceptance criteria

- The doctor selects a registered patient.
- The doctor enters a diagnosis and prescription.
- The doctor selects a consultation type: standard, long, or follow-up.
- On submission, the consultation record is saved.
- The saved record shows the patient, doctor, diagnosis, prescription, consultation type, and date/time.

A User Story Card

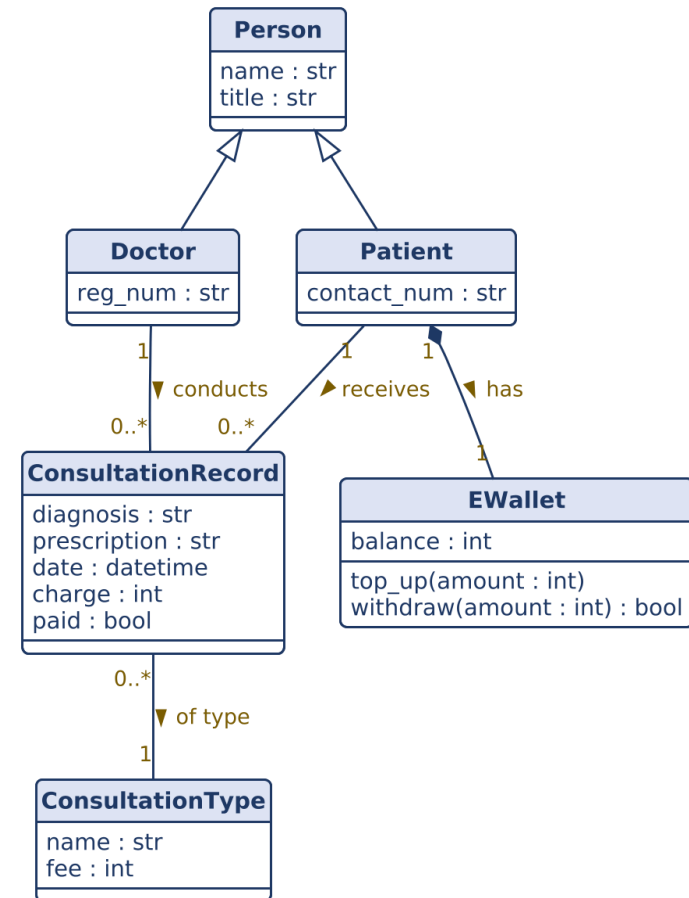
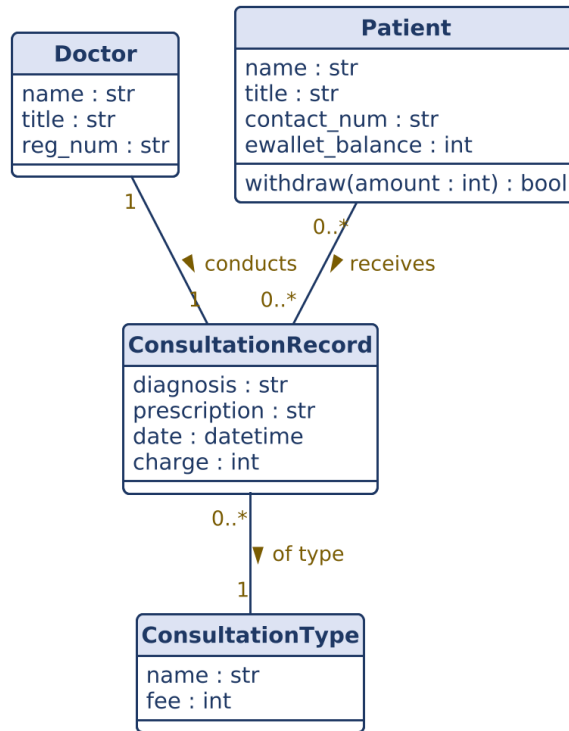
As a **clinic administrator**, I want the appropriate fee to be charged to the patient's e-wallet when a consultation is recorded so that payment is collected automatically.

Acceptance criteria

- The fee is determined from the selected consultation type.
- When the consultation is submitted, the fee is deducted from the patient's e-wallet.
- The consultation record stores the charged fee and whether payment is complete or outstanding.
- When the wallet balance is insufficient, no partial deduction is made.
- The consultation remains saved and the fee is marked as outstanding.

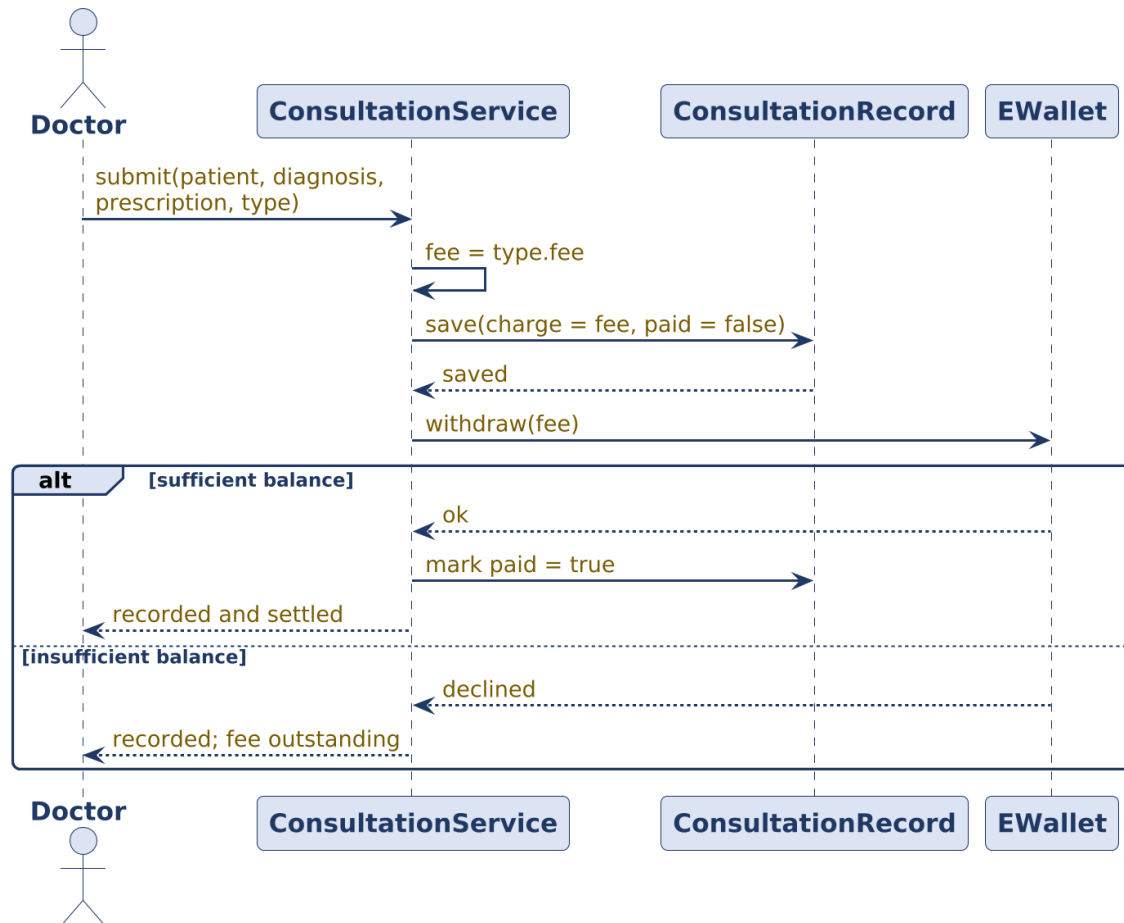
Which Diagram **Best** Models the Relationships?

- Suppose Gemini has generated the **two candidate UML class diagrams** below?
- Which would you choose to scaffold the coding agent?



The Consultation Flow as a Sequence Diagram

- The same story as before — but now the **ordering** and the **insufficient-balance branch** are explicit



What to notice

- The record is saved **before** the debit is attempted — a failed payment never loses clinical data
- The **alt** box makes the failure branch part of the spec, not an afterthought
- For this story it is a **better agent spec** than the class diagram: it pins the ordering
- Still not everything — what if the debit succeeds but marking **paid** fails?

Live Demo – Generating a Prototype with AI

- Feed the agent the **user stories + UML diagrams (PlantUML)** as the spec/context
- For this demo, we generate a **rapid prototype** (e.g. for stakeholder feedback), so we instruct the agent to use an **SQLite in-memory database**
 - easy to run locally; self-contained
 - no authentication, integration with other parts of the system (etc.)
- **Option #1:** prepare a prompt that consolidates the user stories, UML diagrams, and any other context for scaffolding
- **Option #2:** place the user stories, UML diagrams, and agent instructions (e.g. CLAUDE.md, AGENTS.md) **in the project repo** and let the agent inspect the context
 - *Shorter prompts; context stays versioned with the code*
 - *Closer to real-world agentic development*

Full prompt on eLearn

The Other Direction: Code → Diagram

- We just went **spec** → **code**. The reverse (**code** → **spec**) is just as useful:
- reverse-engineer a **class diagram from existing code** to review & understand it
 - onboarding to an unfamiliar module
 - reviewing what an agent generated (e.g. “performative” abstractions 😊)
 - seeing structure the code hides
- **Tools:** your IDE’s diagram *view*
 - *pyreverse* (*Pylint*)
 - or ask an AI to “draw this module as a **PlantUML** class diagram”
- **Caution:** a generated diagram inherits the code’s flaws: read it critically

Git – Collaborating on Code



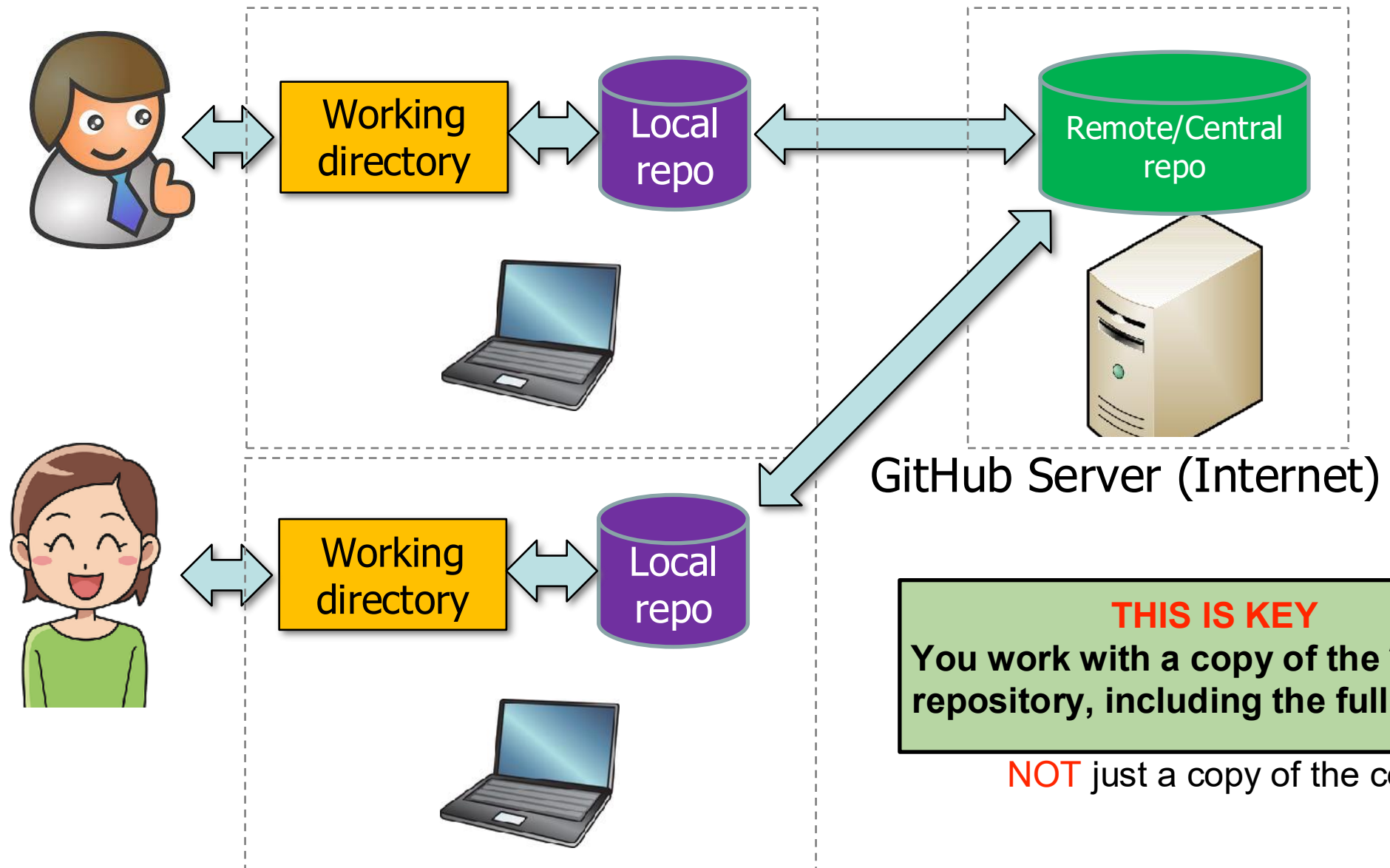
Git – Refresher

- **Git** is a distributed **version control system**
- Provides mechanisms for **sharing changes** to source code and **solving merge conflicts**
- Provides **full control** over when and what you 'publish' to the central code repository
 - *Experiment without disrupting the 'main' codebase*
- Systematically tracks **history of changes**
 - *Allows you to revert to an earlier version*

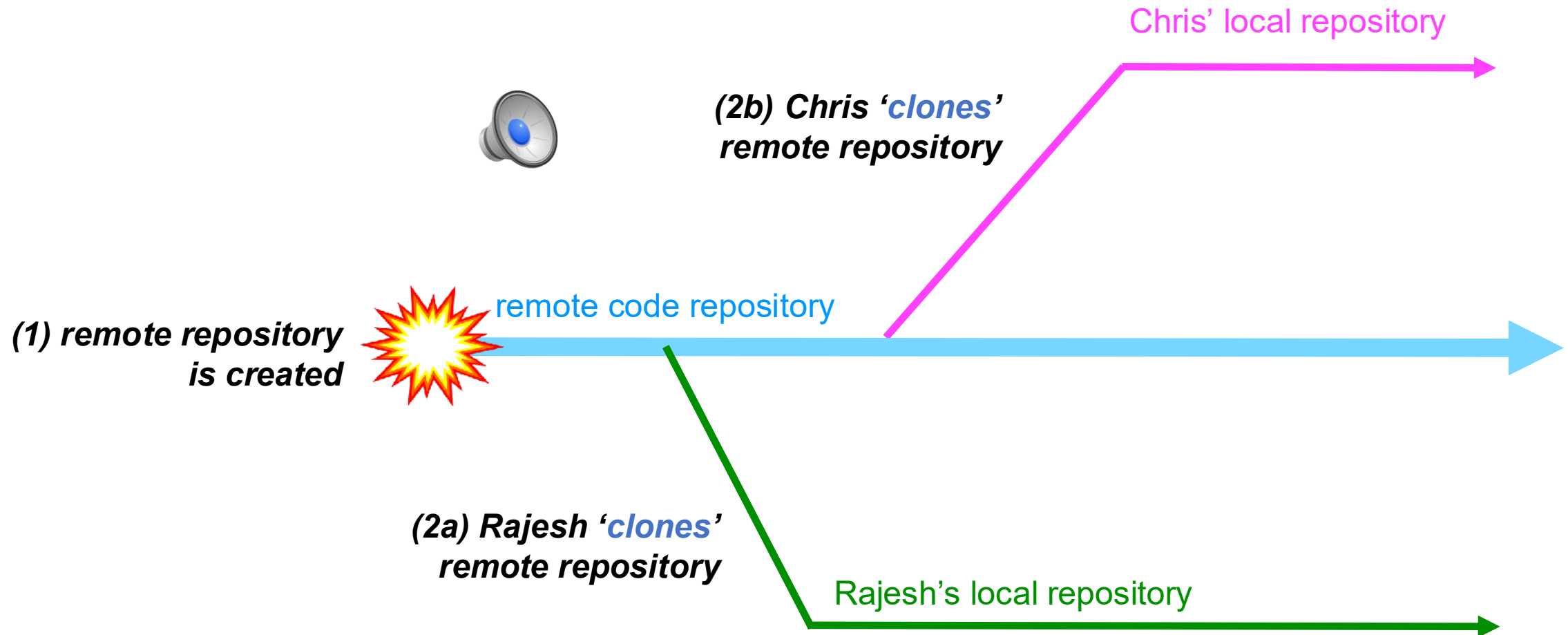


[Linus Torvalds](#)

Git – *it's distributed!*



Key Steps – Setting up



Key Steps – Setting up

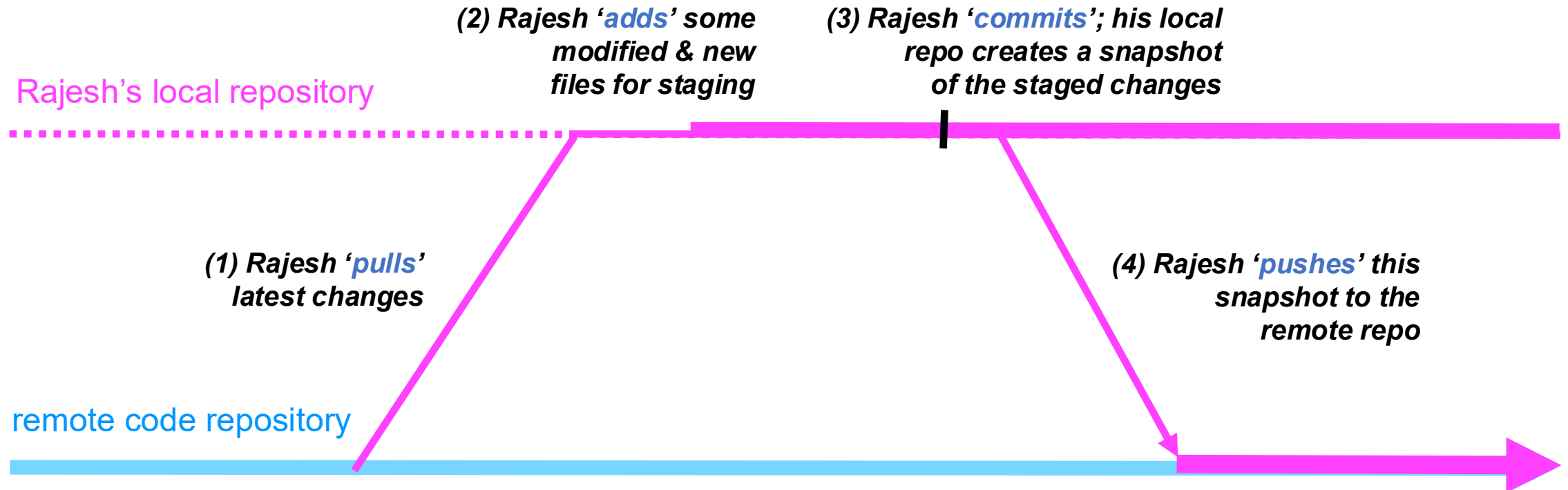


- Create a new **remote/central repository**
- Host on external server, e.g. GitHub

git clone

- Get a **full local copy** of the central repository
- i.e. a history of every version of every file
- Executed **once**, at the start of (your participation in) the project
- CLI: **git clone** *<url>*

Key Steps – Making changes



Key Steps – Making changes

git pull

- Fetches changes from the remote and **integrates the upstream branch** into your current branch
- CLI: **git pull**

git add

- Marks **files to be staged** for the next commit
- Can be **new files** and/or existing **modified** ones
- CLI: **git add** *<file>* or **git add** *<path>*

Key Steps – Making changes

git commit

- Creates snapshot of project's staged changes
- Recorded in local repo; not yet remote one
- CLI: `git commit -m "<message>"`

git push

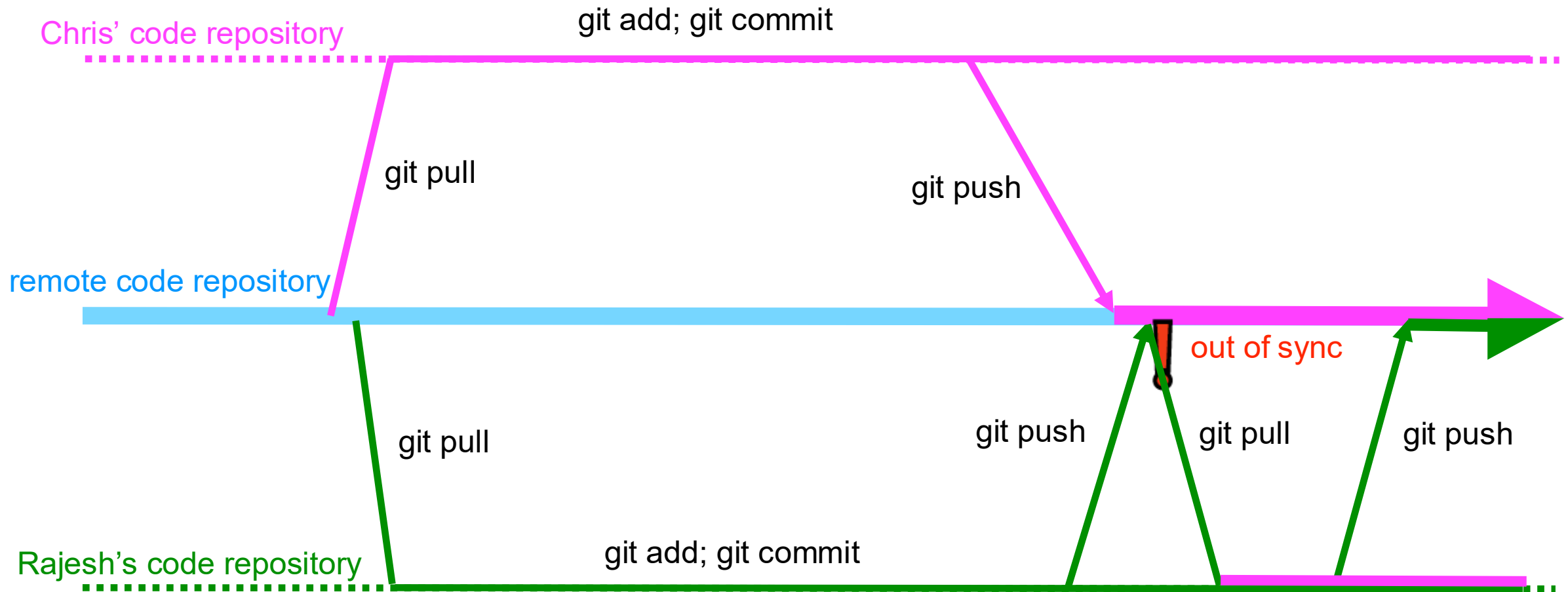
- All committed changes in the locally created snapshot are sent to the remote repo
- Your changes can then be pulled by others
- CLI: `git push`

Conflict Management

- Git's **conflict management** approach is one of the key reasons for its popularity
- Protects against **coordination errors**
 - *e.g. two devs changing the same line of code*
- Facilitates **safe decentralised editing** and efficient **code merging**
- **However** – the absence of conflicts doesn't mean that changes won't break code!
(Solved in Week 6)



Git Automatically Merges Commits – *if it's possible!*



Fixing Conflicts

- If Git **cannot** automatically merge in remote commits, you will be asked to resolve them **directly** in the source code
 - The two versions are **marked in the code**; replace the markers with the correct code

```
def example_function():  
<<<<<< HEAD  
    print("This is my local change.")  
=====  
    print("This is the remote change from origin.")  
>>>>>> abc1234
```



```
def example_function():  
    print("This is my merged change.")
```

- After saving the changes, remember to stage, commit, and push
 - git add, git commit, git push

Collaborating with Git

– feature branches, pull requests, & more –

Branching

- In our Git refresher, we've assumed that every repository consists of a single “path”, i.e. a **single continuous unbroken history** of changes

Chris' local repo – **main** branch



remote repo – **main** branch



Rajesh's local repo – **main** branch



- In practice, **each repository** can itself contain **multiple diverging versions** of the codebase called “**branches**”

Branching – creating a branch

remote repository

main branch

(1) Rajesh '*pulls*' from
remote repo

Rajesh's local repository

main branch

(2) Rajesh creates
new '*branch*'

featureX branch

Branching – modifying/merging branches

remote repository

main branch

featureX branch

(4) finally, featureX is **merged**
into 'main' then deleted

Rajesh's local repository

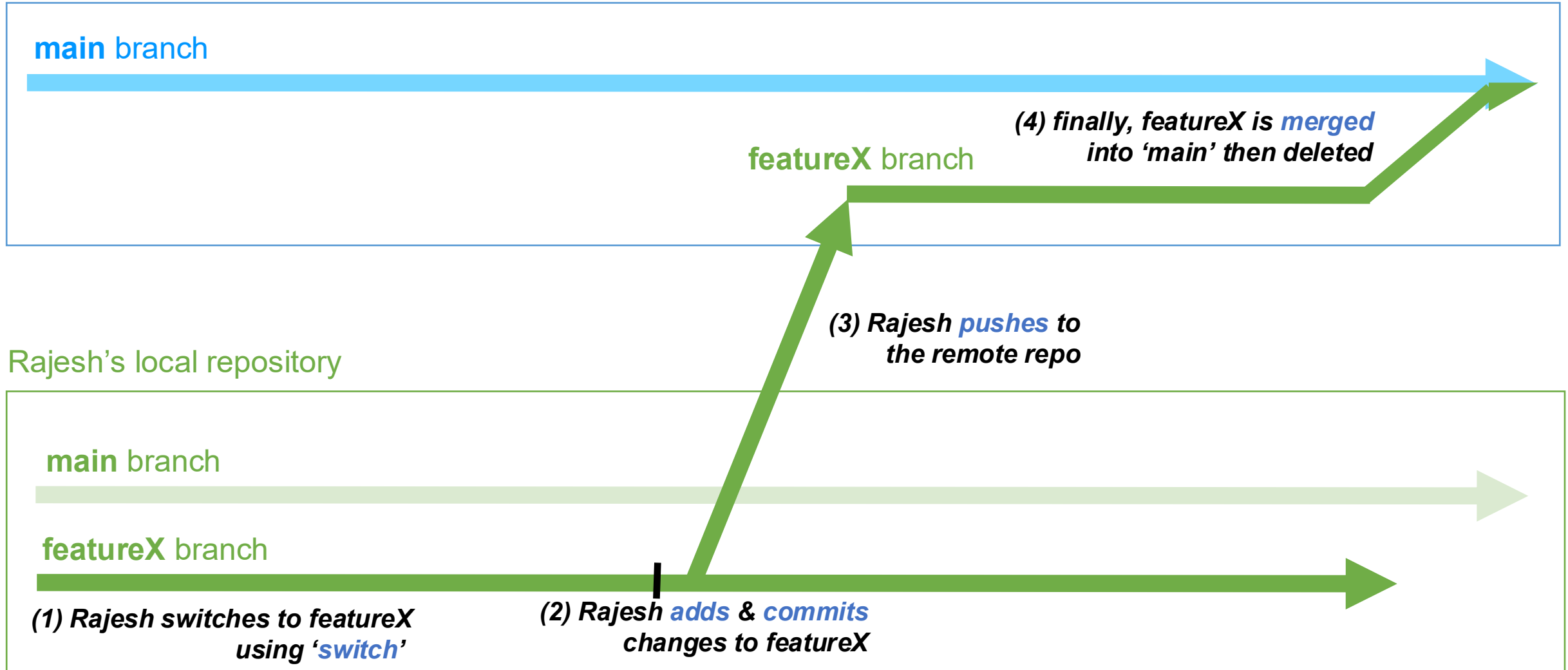
main branch

featureX branch

(1) Rajesh switches to featureX
using '**switch**'

(2) Rajesh **adds** & **commits**
changes to featureX

(3) Rajesh **pushes** to
the remote repo



Rationale for Branching

- Speed and agility are crucial when developing modern software
- Teams need to implement multiple changes at once
- Branching allows you to work on diverging versions of the project without impacting the main branch (usually the stable or production version)
- When the work is complete (and tested to be working), the branch will then be merged back into the main branch
- Git facilitates lightweight and fast branching



main



merge branches



featureX



main

Demo!

Branching commands

git switch

To create a new branch

- CLI: `git switch -c "<branch name>"`

To switch to another branch

- CLI: `git switch "<branch name>"`

git branch

To list all branches

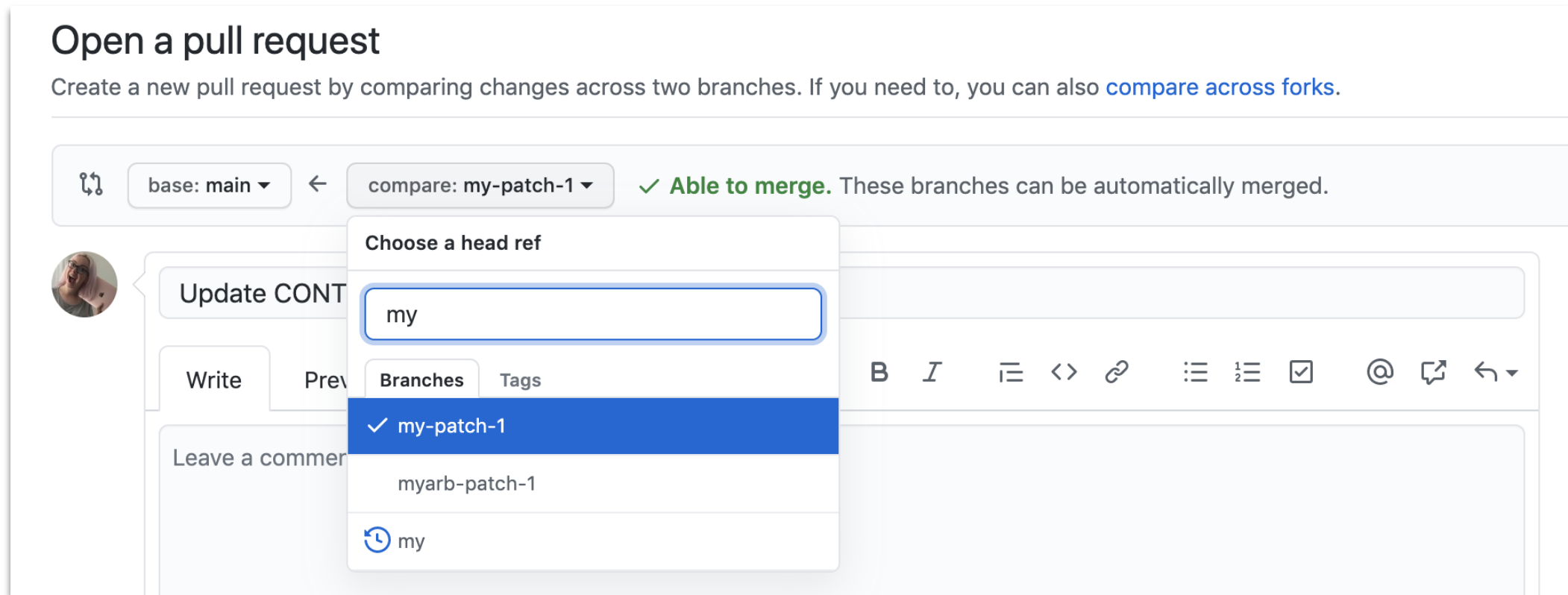
- CLI: `git branch`

Pull Requests (aka Merge Requests)

- **Pull Requests** allow you to tell others about changes in a branch that you would like to “pull” (merge) into the main branch
- Once opened, you can discuss and review the potential changes before they are merged into the main branch
 - both human and AI reviewers leave comments
- Pull requests can trigger automated checks to catch problems before changes are merged into main (**supporting CI – Week 6!**)
- Pull requests can be linked to “issues” to show a bug fix is in progress, and to automatically close the issue when someone merges the pull request



Pull Requests on GitHub



Source: [GitHub Docs](#)

What Makes a Good Pull Request (PR)?

- A PR is the **unit of collaboration** – the thing your teammates should actually review

A good PR...

- does **one coherent thing** – not three unrelated fixes bundled together
- is **small enough to review properly** – a reviewer loses the thread past a few hundred lines
- is **linked to a story or task** so the **why** is traceable
- has a description explaining **why**, not just **what** – the diff already shows what
- arrives with **checks green** (Week 6), and gets **merged quickly** rather than sitting for days
 - a PR left open for a week is a merge conflict waiting to happen

Collaboration Strategies

Collaboration Strategies

- Branches can facilitate different development strategies in Git, for example:
- **Feature Branches** – each new feature/story gets its own branch, and is merged back into main once completed and tested
 - *Work is developed on separate branches and merged into main through PRs*
 - *Can develop new feature in “blissful isolation”...*
 - *... but **long-lived feature branches** increase integration **risk***
- **Release Branches** – create a dedicated branch for the next software release, and thoroughly test it in isolation from features being developed for subsequent releases

How Long Before we Integrate?

Optimise for **individual** productivity

vs.

Optimise for **team** productivity

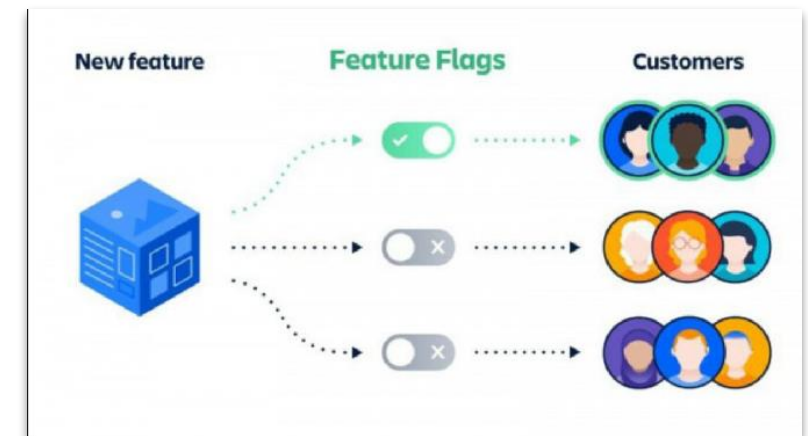
- Developers/agents work independently for **extended periods**
 - Changes accumulate into **large batches**
 - Feedback on integration is **delayed**
 - Merges become **harder and riskier**
-
- Everyone integrates into a **shared trunk frequently**
 - Direct commits or **short-lived branches**
 - Changes remain in **small batches**
 - CI detects problems **quickly**
 - An undetected bug can **break the project**

Trunk-Based Development

- Working in a **common branch** is known as **trunk-based development**
- Developers **integrate into main frequently** – usually at least daily – either directly or through **very short-lived branches/PRs**
- **Rationale:** integrating small, frequent changes is **less likely to lead to conflicts** than large batches of changes developed over several days in isolated branches
 - ...and if a conflict occurs, it's **easier to resolve** (since the changes were smaller)
- Trunk-based development poses **some additional challenges**:
 - *If bugs are introduced to main, they must be caught/fixed ASAP to avoid delaying work*
 - *What to do about **partially finished** large new features? **Do we commit partial work to main?***

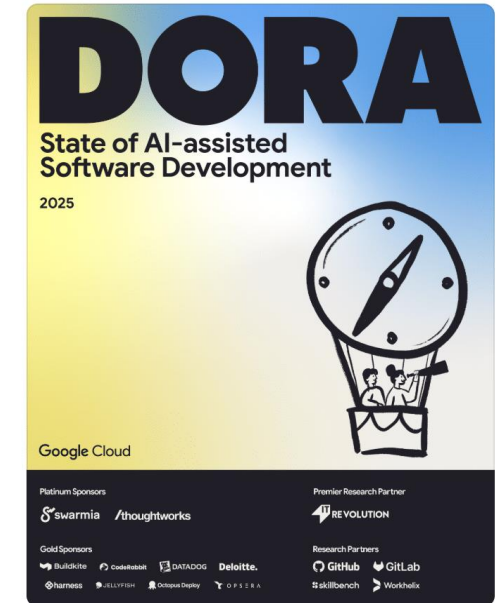
Feature Flags in Trunk-Based Development

- **Feature flags** allow developers to wrap new changes (e.g. **partially-finished features**) in an inactive code path and activate it later
 - e.g. wrapping app logic / UI elements with a **conditional statement**
 - **Enabled** or **disabled** based on a configuration setting / environment variable
- Removes the need to develop a large new feature in a long-lived feature branch
- Remember to remove the feature flags when no longer needed!
 - *Otherwise can accumulate into technical debt*



AI Can Raise the Cost of Delayed Integration

- *DORA 2025 – State of AI-assisted Software Development* – based on responses from ~5,000 tech professionals; 90% now use AI at work
- Suggests that **AI-assisted development** increases throughput, but may put **stability** under pressure
 - **Strong automated testing** and fast feedback help contain the risk
 - **Small batches, frequent integration** and mature version control amplify the benefits of AI
 - **Trunk-based development** provides a practical foundation for these practices



For Your IS212 Project

Recommended workflow

- use **short-lived branches** for small, coherent changes – usually one story or task
- open a **small PR**; get a teammate's **review**; let the **checks pass**
- merge into a **protected main**; then delete the branch
- sync frequently – **don't** keep a branch open for a whole sprint
- Small batches + review + green checks → **fewer painful merges**, and a main that **stays releasable**

Agent Collaborators

Collaborating in the AI Era

- Your collaborators may now include **agents that open their own branches & PRs**
- So the fundamentals matter **more**, not less:
 - small reviewable PRs · clear commit messages · short-lived branches
- **AI reviewers** (Copilot, CodeRabbit...) can automatically review every PR – broad coverage, but **humans still own judgement**
- **Handle agent files deliberately:**
 - ignore local state: caches, transcripts, temporary plans, machine settings, credentials
 - commit intentionally: shared team instructions & project rules (**CLAUDE.md**, **.cursor/rules**)
- AI can draft commit messages & resolve conflicts; **you own correctness**

A Workflow with Agent Collaborators

- The same protocol, with an agent doing the implementing:
 - human writes the Jira task → agent works on a short-lived branch → agent opens a PR
 - → CI runs the checks → AI reviewer comments → human reviews behaviour & design → merge

So what actually changed?

- Surprisingly little about the collaboration protocol itself
- But small changes, clear specifications, tests, review and traceability all matter more, because code arrives faster than anyone can read it
- The human stays accountable for what lands on main — authorship is not the same as responsibility

The Repository as Shared Context

- The repository holds **more than code** — it is the team's shared context:
 - README · architecture & design docs · tests · coding conventions
 - issue & PR templates · CI configuration · **agent instructions**
- Anything a **new teammate** would need to work well here, an **agent** needs too
- Agent instruction files are just the newest member of that family:
 - **AGENTS.md** · **CLAUDE.md** · **.cursor/rules** · **.github/copilot-instructions.md**
- Committed, reviewed and versioned **like code** — which is why we **commit** them rather than ignore them

Good Git Habits (Human & Agent)

- Write meaningful commit messages: often the only thing reviewers (and agents) read
- Sync with the remote before you start, and before opening/updating a PR – avoid drift (rather than a rote ‘pull before push’)
- Never commit secrets – keys in history stay in history; pass them via env vars
- Keep the repo tidy: .gitignore junk, modular structure, agent context, a real README
- Don't commit as someone else, zip up sources, or split repos to “avoid conflicts”

Lab & Reflections

Week 5 Lab

- This week's lab will guide you through generating **UML class diagrams** for a novelty scenario
- It will also take you through the pull-request-based **Git workflow** from class
- Formative exercises (no submission)



Week 5 Lab – From Class Diagram to Pull Request

This week we take one small system the whole way: from a user story and a UML class diagram, to a working prototype an agent builds for you, to a Git repository with a protected `main` branch, and then we handle the change request that always arrives the moment you think you are done. Producing each of these artifacts is cheap now. Keeping them consistent with one another, and deciding what “done” means, is the part that still belongs to you.

A note on the diagram. We keep it as **PlantUML text** (not a picture) for the same reason we kept the C4 model as a DSL last week: text is diffable in a pull request, editable line by line, and regenerated when the design changes. By the end of the lab your diagram will live in the repository next to the code it describes, and you will change both in the same pull request.

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 🖥️
- <http://smu.sg/is212-reflections-2026>

